ACADÉMIE DE PROGRAMMATION — IFRI
Bloc 2 · Module 2

Document d'Architecture Logicielle

Site DevStart Agency — Modularisation & Organisation des fichiers

GROUPE 3

★ Chef de groupe : Faridath GABA ISSIFOU

Mawoussi Sandrine AGBODJI
N'DA Prielle Arifath
Fréjus ZANKPO
Abdou-Hakim KARIM ISSAOU
Ivan Claudel Idjäm OGA

Date : 30 avril 2026 · L1 IFRI · Parcours Développement Web

Table des matières

1. Introduction — Contexte et problème posé	3
2. Cartographie de la page existante	4
3. Architecture cible — Structure de dossiers	5
4. Tableau des fichiers CSS	6
5. Éléments réutilisables (composants)	7
6. Stratégie d'évolution — Pages futures	8
7. Conclusion	9

1. Introduction — Contexte et problème posé

1.1 Présentation du projet

DevStart Agency est une agence web fictive dont nous avons analysé et documenté la page d'accueil lors du Module 1. Lors de ce premier module, le code hérité d'un stagiaire se composait de deux fichiers uniques : un fichier index.html contenant toutes les sections de la page, et un fichier style.css regroupant l'ensemble des règles CSS sans aucune séparation.

Cette approche, bien que fonctionnelle pour une petite page, devient rapidement ingérable dès lors que le projet prend de l'ampleur. La direction de DevStart Agency souhaite désormais faire évoluer le site en y ajoutant plusieurs nouvelles pages : un Portfolio, un Blog et un Espace Client.

1.2 Le problème identifié

L'analyse du code du stagiaire a mis en évidence plusieurs problèmes architecturaux majeurs :

- Un seul fichier CSS de 298 lignes couvre 7 sections distinctes sans délimitation claire
- Les classes CSS sont nommées de façon cryptique (.b1, .c, .d...) sans lien sémantique avec leur contenu
- Toute modification d'une section risque d'impacter involontairement une autre section
- L'ajout d'une nouvelle page nécessiterait de fouiller l'intégralité du fichier CSS
- Aucun élément réutilisable n'est identifié ou isolé

1.3 Notre mission

Notre mission dans ce Module 2 est de concevoir et d'implémenter une architecture logicielle modulaire pour le site DevStart Agency, en appliquant le principe fondamental de responsabilité unique : chaque fichier ne doit gérer qu'une seule partie du site. Nous ne modifions pas le design — nous organisons le code de façon professionnelle.

Situation initiale (Module 1)	Situation cible (Module 2)
1 fichier HTML — tout en vrac	1 fichier HTML structuré et commenté
1 fichier CSS de 298 lignes — tout mélangé	9 fichiers CSS à responsabilité unique
Classes nommées .b1, .c, .d...	Structure lisible par n'importe quel développeur
Évolution difficile — tout est couplé	Architecture extensible pour de futures pages

2. Cartographie de la page existante

Avant de réorganiser quoi que ce soit, nous avons dressé la carte complète du site actuel. Voici l'inventaire exhaustif de chaque section, des éléments qu'elle affiche, de ses besoins et des classes CSS qui lui sont associées.

Section	Ce qu'elle affiche	Ce dont elle a besoin	Classes CSS utilisées	Fichier CSS cible
Header / Nav	Logo DevStart + liens navigation (acc, nous, serv, cont)	Liens hypertextes, fond sombre, layout flex	.b1, .b1a, .b1b, .b1b a, :hover	header.css
Hero	Titre principal, description agence, bouton CTA, image hero	hero.png, disposition flex, bouton coloré	.c, .c1, .c1a, .c1b, .c1c, .c2, img	hero.css
Services	Titre de section + 4 cartes (Design, Dev, Marketing, Maintenance)	Icônes emoji, grille flex, fond gris clair	.d, .d0, .dd, .d1-.d4, .d1i-.d4i, .d1t-.d4t, .d1p-.d4p	services.css
About	Image équipe + texte présentation + bouton	team.png, layout flex, bouton sombre	.e, .e1, .e2, .eimg, .e2t, .e2p, .e2b	about.css
Témoignages	Titre + 3 cartes de témoignages clients	Fond sombre, bordure rouge, 3 citations	.f, .f0, .ff, .f1-.f3, .f1t-.f3t, .f1n-.f3n	testimonials.css
Contact	Titre + formulaire (4 champs + bouton)	Champs input, textarea, bouton submit	.g, .g0, .gf, .gi, .gt, .gb, :hover	contact.css
Footer	Copyright + liens réseaux sociaux + email	3 colonnes flex, fond très sombre	.h, .h1, .h2, .h2 a, .h3, :hover	footer.css

2.1 Règles CSS globales identifiées

Lors de l'analyse du fichier style.css, nous avons identifié les règles qui ne sont propres à aucune section mais s'appliquent à l'ensemble du site. Ce sont les premières règles du fichier :

Règle CSS	Effet	Affectation
* { margin: 0; padding: 0; }	Supprime les marges par défaut du navigateur	base.css
body { font-family: Arial; color: #333; }	Définit la police et la couleur de texte par défaut	base.css
#1a1a2e — bleu foncé principal	Couleur de marque — header, titres, boutons	base.css (variable)
#e94560 — rouge accent	Couleur d'accent — boutons CT	base.css (variable)

	A, bordures	

3. Architecture cible — Structure de dossiers

Sur la base de la cartographie réalisée, nous avons conçu la structure de dossiers suivante. Chaque fichier a une responsabilité unique et clairement définie.

3.1 Vue d'ensemble de l'arborescence

```
devstart-agency/
│
├── index.html              ← Page d'accueil
│
├── styles/                 ← Tous les fichiers CSS
│   ├── base.css            ← Reset + variables globales
│   ├── header.css          ← Navigation et logo
│   ├── hero.css            ← Section d'accroche
│   ├── services.css        ← Cartes de services
│   ├── about.css           ← Présentation équipe
│   ├── testimonials.css    ← Témoignages clients
│   ├── contact.css         ← Formulaire de contact
│   ├── footer.css          ← Pied de page
│   └── components.css      ← Boutons réutilisables
│
├── assets/                 ← Ressources statiques
│   ├── hero.png
│   └── team.png
│
└── README.md               ← Documentation du projet
```

3.2 Justification de l'organisation

Le dossier styles/ regroupe tous les fichiers CSS car ils partagent la même nature — ils définissent l'apparence du site. Les regrouper dans un dossier dédié permet à tout développeur rejoignant le projet de savoir immédiatement où aller pour modifier un style.

Le dossier assets/ centralise toutes les ressources graphiques (images, icônes futures). Cette séparation garantit que le HTML ne référence jamais des fichiers éparpillés à la racine.

Le fichier index.html reste à la racine car c'est le point d'entrée standard de tout site web — les serveurs web cherchent ce fichier en premier par convention.

3.3 Liens dans le HTML

Après réorganisation, le fichier index.html devra référencer les fichiers CSS dans cet ordre précis, car base.css doit être chargé en premier pour que les variables et le reset s'appliquent avant les styles spécifiques :

```
<link rel="stylesheet" href="styles/base.css">
<link rel="stylesheet" href="styles/header.css">
<link rel="stylesheet" href="styles/hero.css">
```

```
<link rel="stylesheet" href="styles/services.css">
<link rel="stylesheet" href="styles/about.css">
<link rel="stylesheet" href="styles/testimonials.css">
<link rel="stylesheet" href="styles/contact.css">
<link rel="stylesheet" href="styles/footer.css">
<link rel="stylesheet" href="styles/components.css">
```

4. Tableau des fichiers CSS

Ce tableau constitue la référence principale de notre architecture. Chaque fichier CSS prévu est décrit avec sa responsabilité unique, les classes qu'il contient et le nombre de lignes estimé après découpage.

Fichier	Responsabilité unique	Éléments gérés	Lignes	Priorité
base.css	Définir les règles globales et le reset CSS du site	Sélecteur *, body (font, couleur), variables de couleurs (#1a1a2e, #e94560, #f9f9f9)	~12	1 — Critique
header.css	Gérer l'apparence de la barre de navigation	.b1 (layout flex, fond), .b1a (logo), .b1b (liens nav), états hover des liens	~25	2 — Haute
hero.css	Styliser la section d'accroche principale	.c (flex, padding), .c1, .c1a (titre), .c1b (description), .c1c a (bouton CTA), .c2, img	~42	2 — Haute
services.css	Afficher les cartes de services en grille	.d (padding), .d0 (titre), .dd (flex), .d1 à .d4 (cartes), icônes, titres, descriptions des cartes	~45	3 — Normale
about.css	Présenter la section équipe	.e (flex, fond), .e1, .eimg (image arrondie), .e2, .e2t (titre), .e2p (paragraphes), .e2b a (bouton)	~46	3 — Normale
testimonials.css	Styliser les témoignages clients	.f (fond sombre), .f0 (titre blanc), .ff (flex), .f1 à .f3 (cartes), textes et auteurs italiques	~42	3 — Normale
contact.css	Mettre en forme le formulaire de contact	.g (fond), .g0 (titre), .gf (flex column), .gi (inputs), .gt (textarea), .gb (bouton), états hover	~55	3 — Normale
footer.css	Styliser le pied de page	.h (flex, fond très sombre), .h1 (copyright), .h2 a (réseaux sociaux), .h3 (email), états hover	~32	3 — Normale
components.css	Centraliser les éléments d'interface réutilisables	Classe .btn-primary (bouton rouge), .btn-secondary (bouton sombre) — partagés entre sections et futures pages	~20	4 — Future

Total estimé : ~319 lignes réparties sur 9 fichiers, contre 298 lignes dans un seul fichier. La légère augmentation s'explique par l'ajout de commentaires d'en-tête dans chaque fichier et la section components.css (nouvelle).

5. Éléments réutilisables (composants)

L'analyse du code existant a mis en évidence des patterns qui se répètent dans plusieurs sections. Ces éléments méritent d'être isolés dans un fichier dédié (components.css) pour éviter la duplication de code et faciliter les modifications futures.

5.1 Les boutons — élément le plus dupliqué

Dans le code actuel, deux types de boutons apparaissent dans deux sections différentes avec des styles quasi-identiques codés en dur :

Bouton	Où il apparaît actuellement	Style actuel (classe)	Classe cible
Bouton rouge (CTA)	Section Hero (.c1ca) + Section Contact (.gb)	background: #e94560 · padding · border-radius · font-weight bold	**.btn-primary**
Bouton sombre	Section About (.e2ba)	background: #1a1a2e · padding · border-radius · font-weight bold	**.btn-secondary**

5.2 Justification de la création de components.css

- Actuellement, si on veut changer le border-radius des boutons, il faut modifier 3 endroits différents dans le CSS — avec components.css, on ne le change qu'une seule fois
- Les futures pages (Portfolio, Blog, Espace Client) auront aussi des boutons — elles pourront utiliser .btn-primary et .btn-secondary sans réécrire les styles
- Si le graphiste décide de changer la couleur principale de #e94560 à une autre teinte, un seul fichier suffit

5.3 Autres candidats à la réutilisation (futures itérations)

En vue des pages futures, ces éléments pourraient également être extraits en composants :

- Les cartes (cards) — le pattern carte avec fond gris, padding et border-radius est utilisé dans Services et Témoignages
- Les titres de section — .d0, .e2t, .f0, .g0 ont tous des styles similaires (bold, grande taille, couleur marque)
- Le layout flex à deux colonnes — utilisé dans Hero et About avec des variantes mineures

6. Stratégie d'évolution — Pages futures

Une bonne architecture doit anticiper la croissance. DevStart Agency prévoit d'ajouter trois nouvelles pages : Portfolio, Blog et Espace Client. Voici comment notre architecture les accueille sans modification structurelle majeure.

6.1 Arborescence projetée à 6 mois

```
devstart-agency/
├── index.html
├── portfolio.html          ← NOUVELLE PAGE
├── blog.html               ← NOUVELLE PAGE
├── espace-client.html      ← NOUVELLE PAGE
├── styles/
│   ├── base.css            ← inchangé — partagé par toutes les pages
│   ├── components.css      ← inchangé — partagé par toutes les pages
│   ├── header.css          ← inchangé — même nav sur toutes les pages
│   ├── footer.css          ← inchangé — même footer sur toutes les pages
│   ├── [fichiers actuels...]  ← inchangés
│   ├── portfolio.css       ← NOUVEAU — spécifique Portfolio
│   ├── blog.css            ← NOUVEAU — spécifique Blog
│   └── auth.css            ← NOUVEAU — spécifique Espace Client
└── assets/
```

6.2 Principe d'extension

Grâce à notre découpage modulaire, ajouter une nouvelle page ne nécessite que deux actions :

- Créer un nouveau fichier HTML à la racine (ex: portfolio.html)
- Créer un nouveau fichier CSS dans styles/ pour les styles spécifiques à cette page (ex: portfolio.css)

Les fichiers base.css, components.css, header.css et footer.css sont déjà partagés. La nouvelle page n'a qu'à les référencer avec des balises <link> — aucune duplication de code.

6.3 Tableau des pages futures

Page	Fichier HTML	Fichier CSS spécifique	Fichiers CSS partagés
Portfolio	portfolio.html	styles/portfolio.css	base + components + header + footer
Blog	blog.html	styles/blog.css	base + components + header + footer
Espace Client	espace-client.html	styles/auth.css	base + components + header + footer

7. Conclusion

7.1 Ce que cette architecture change concrètement

En passant d'un unique fichier style.css de 298 lignes à une architecture modulaire de 9 fichiers à responsabilité unique, nous transformons radicalement la maintenabilité du projet DevStart Agency.

Scenario	Avant → Après
Un développeur veut modifier le style du footer	Cherche dans 298 lignes → Ouvre footer.css directement
Le bouton CTA change de couleur	Modifie 3 classes éparpillées → Modifie 1 ligne dans components.css
Ajout de la page Portfolio	Refactoring complet → Créer portfolio.html + portfolio.css
Un nouveau dev rejoint le projet	Comprend rien → Comprend la structure en 30 secondes

7.2 Le principe retenu

Ce module nous a enseigné que la qualité d'un code ne se mesure pas uniquement à son fonctionnement, mais aussi à sa lisibilité et à sa capacité à évoluer. Une architecture modulaire ne ralentit pas le développement — elle l'accélère sur le long terme.

Le principe d'une responsabilité par fichier peut sembler contraignant au départ. Mais il garantit que chaque modification est localisée, que les bugs sont faciles à trouver, et que le projet peut grandir sans devenir ingérable.

7.3 Ce que nous retenons

- Penser avant de coder — un schéma d'architecture épargne des heures de refactoring
- Nommer clairement — un fichier dont le nom dit ce qu'il fait vaut mieux que des commentaires
- Prévoir la croissance — une bonne architecture d'aujourd'hui ne doit pas bloquer demain
- La modularité est un investissement — coûteuse à mettre en place, rentable à maintenir

> *"La qualité d'une architecture se mesure à la facilité avec laquelle on peut y ajouter quelque chose de nouveau."*
>
> — Cahier des charges Bloc 2, Module 2